

Практическая работа «Распознавание изображений с помощью искусственного интеллекта»

Цель работы: освоить основы распознавания изображений с использованием технологий искусственного интеллекта, научиться применять готовые инструменты или библиотеки для анализа и классификации изображений.

Порядок выполнения задания, методические указания:

- 1- изучить ряд теоретических положений;
- 2- решить ряд практических заданий (экранные формы и листинг программного кода с комментариями);
- 3- сформулировать вывод.

Содержание отчета: титульный лист, предоставить решение выбранных заданий, выводы, ответы на вопросы.

Теоретическая часть

Распознавание изображений является одной из ключевых задач в области искусственного интеллекта и компьютерного зрения. С развитием вычислительных мощностей и появлением больших наборов данных стало возможным создание высокоэффективных моделей, способных автоматически анализировать визуальную информацию.

Распознавание изображений включает в себя процесс обработки и интерпретации визуальных данных с целью выявления объектов, их характеристик и взаимосвязей.

Основные этапы включают:

- 1- предварительную обработку изображений;
- 2- извлечение признаков;
- 3- классификацию или детекцию объектов.

Современные системы распознавания базируются на нейронных сетях, которые имитируют работу человеческого мозга. Наиболее эффективными являются сверточные нейронные сети (CNN), способные автоматически выделять важные признаки изображения.

Анализ данных занимает важное место в современной науке и прикладных исследованиях. Для обучения и тестирования методов анализа широко применяются стандартные наборы данных. Одним из таких наборов является набор данных «Ирисы Фишера» (Fisher's Iris Dataset), предложенный британским статистиком Рональдом Фишером в 1936 году. Он используется для демонстрации методов дискриминантного анализа, классификации и визуализации данных. Набор данных был опубликован Р. А. Фишером в статье «The Use of Multiple Measurements in Taxonomic Problems». Исходные измерения были получены ботаником Эдгаром Андерсоном и использовались для изучения морфологических различий между тремя видами ирисов.

Набор данных состоит из 150 наблюдений, относящихся к трём видам ирисов: Iris

setosa, *Iris versicolor*, *Iris virginica*.

Для каждого экземпляра измерены четыре количественных признака: длина чашелистика (sepal length), ширина чашелистика (sepal width), длина лепестка (petal length), ширина лепестка (petal width). Каждый вид представлен 50 наблюдениями, что делает набор данных сбалансированным.

В рамках анализа набора данных применяются следующие методы:

- 1- описательная статистика (среднее значение, медиана, стандартное отклонение);
- 2- корреляционный анализ признаков;
- 3- визуализация данных с использованием диаграмм рассеяния;
- 4- методы классификации (линейный дискриминантный анализ, k-ближайших соседей, деревья решений).

Описательный анализ показывает, что вид *Iris setosa* значительно отличается от двух других видов по размерам лепестков, что упрощает задачу классификации. Между признаками длины и ширины лепестков наблюдается высокая корреляция. Виды *Iris versicolor* и *Iris virginica* имеют частично пересекающиеся распределения признаков, что делает их разделение более сложным.

Применение методов классификации демонстрирует высокую точность распознавания видов, особенно при использовании признаков лепестков. В большинстве случаев точность классификации превышает 90%.

Популярность набора данных «Ирисы Фишера» обусловлена его простотой, наглядностью и хорошо изученными свойствами. Он широко применяется в образовательных целях, а также для первичной проверки новых алгоритмов анализа данных. Несмотря на ограниченный объём, набор данных остаётся актуальным инструментом для демонстрации базовых концепций статистики и машинного обучения.

Набор данных «Ирисы Фишера» является классическим примером структурированных данных, подходящих для изучения методов анализа и классификации. Его использование позволяет эффективно освоить основные этапы анализа данных — от первичной обработки до интерпретации результатов. Практическая и образовательная ценность данного набора данных сохраняется и в настоящее время.

Практическая часть

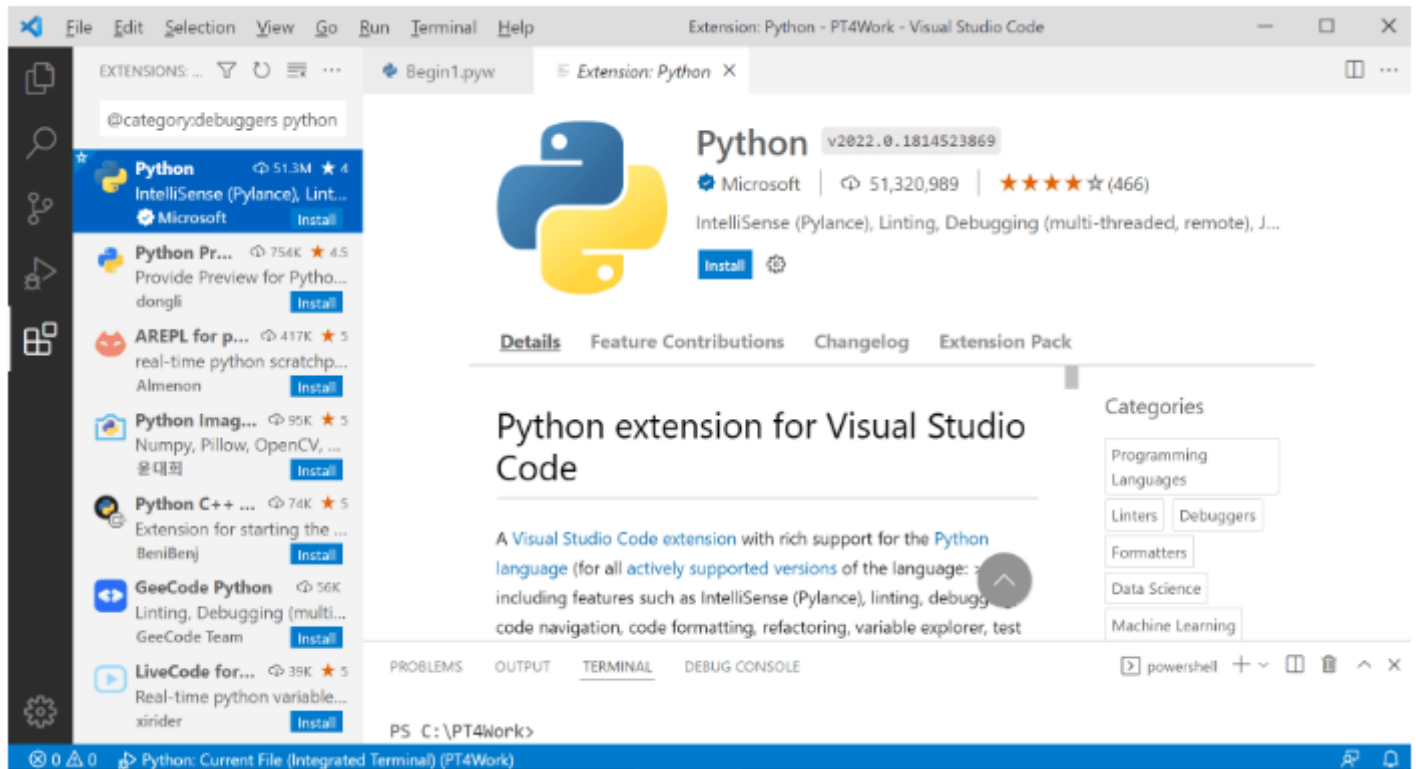
Задание 1. Установить и настроить VS Code и Python

Установить базовый дистрибутив языка Python одной из версий (например, [3.7](#), [3.8](#), [3.9](#)). При установке обратите внимание на каталог, в котором будет размещена система Python (по умолчанию система Python последних версий устанавливается в подкаталог каталога, связанного с текущим пользователем, например, `C:\Users\<UserName>\AppData\Local\Programs\Python\Python39`). В ходе установки выберите все предлагаемые компоненты.

2. Установить среду VS Code <https://code.visualstudio.com/download#>, вариант **Windows, 64 bit, User Installer** или **System Installer** (при выборе **System Installer** среда устанавливается в каталоге `C:\Program Files`, при выборе **User Installer** —

в подкаталоге каталога, связанного с текущим пользователем, например, C:\Users\<UserName>\AppData\Local\Programs\Microsoft VS Code).

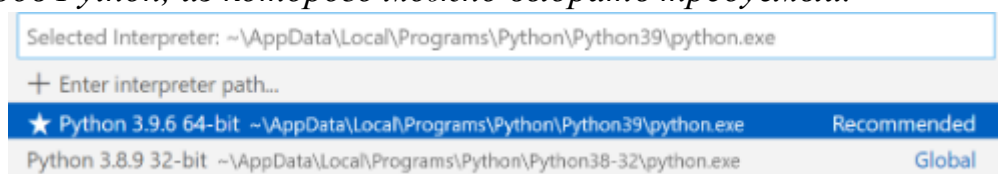
3. В результате в левой панели редактора VS Code появится список доступных расширений для языка Python, из которого надо выбрать вариант **Python (Microsoft)** и нажать связанную с ним кнопку **Install**.



После установки данного расширения появится возможность запускать программы Python из среды VS Code нажатием клавиши [F5].

Примечание 1. Для установки любого расширения можно отобразить в левой части окна VS Code панель *Extensions* со списком доступных расширений (нажав кнопку  или комбинацию клавиш [Ctrl]+[Shift]+[X]), выбрать из списка нужное расширение и нажать связанную с ним кнопку *Install*. Чтобы отобразить список расширений для языка Python, достаточно ввести текст *Python* в поле поиска в верхней части панели *Extensions*.

Примечание 2. После установки расширения для языка Python в редакторе VS Code появится возможность выбирать интерпретатор Python для запуска программ на этом языке. Текущий вариант интерпретатора указывается на статусной панели (в нижней левой части окна редактора), например **Python 3.9.6 64-bit**. Если щелкнуть мышью на этом тексте, то появится список доступных интерпретаторов Python, из которого можно выбрать требуемый.



Задание 2. Установить и настроить платформу Anaconda

1- Загрузить установочный файл с сайта

<https://www.anaconda.com/products/individual#windows>

2- Запустить загруженный файл

3- Следуйте рекомендациям установщика и в конце каждого шага нажимайте **Next**

а) принять лицензионное соглашение, нажав **I Agree**

б) выбрать режим установки **Just Me** (для текущего пользователя). При выборе режима **for all users** (для всех пользователей) потребуются права администратора.

в) выбрать папку установки Anaconda (рис.3). Путь не должен содержать пробелов или символов в кодировке **unicode**.

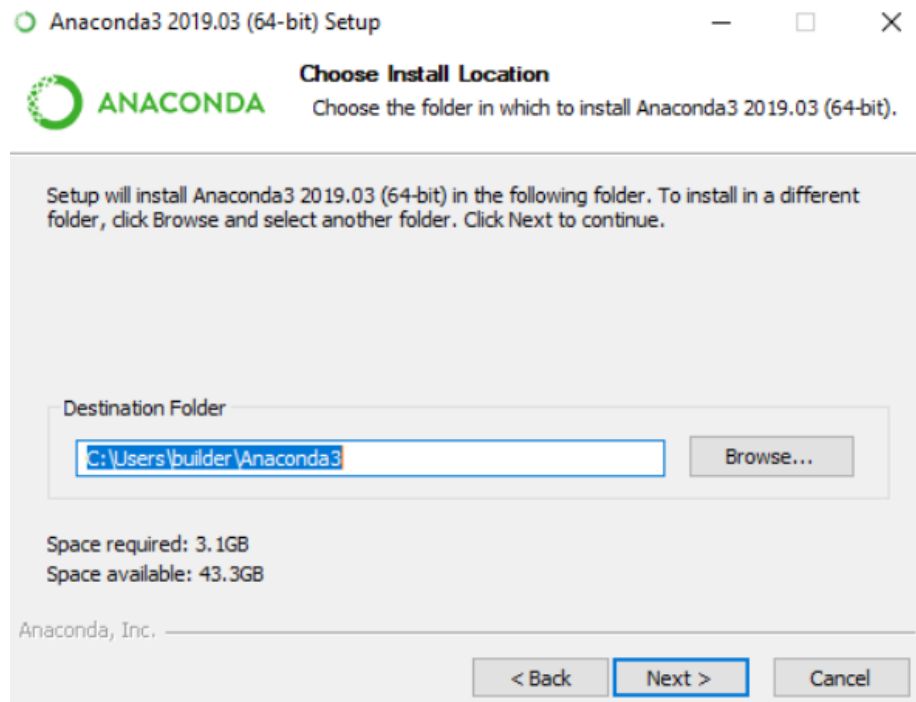


Рис.3. Выбор места установки платформы

г) В следующем окне расширенных опций рекомендуется НЕ включать флаг **add Anaconda to the PATH environment variable**, чтобы не мешать другому ПО (рис. 4).

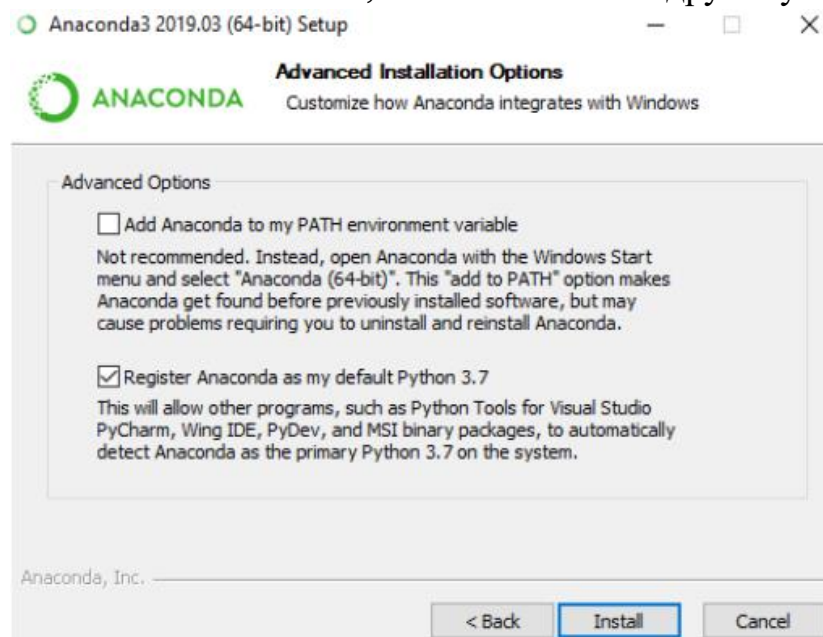


Рис.4. Задание дополнительных параметров установки

д) Выбрать вариант Anaconda в качестве ПО Python по умолчанию.

е) Нажать кнопку **Install**

ж) На следующих шагах нажать **Next**

После успешной установки будет выведено окно (рис. 5). Отключить все флажки и нажать **Finish**.

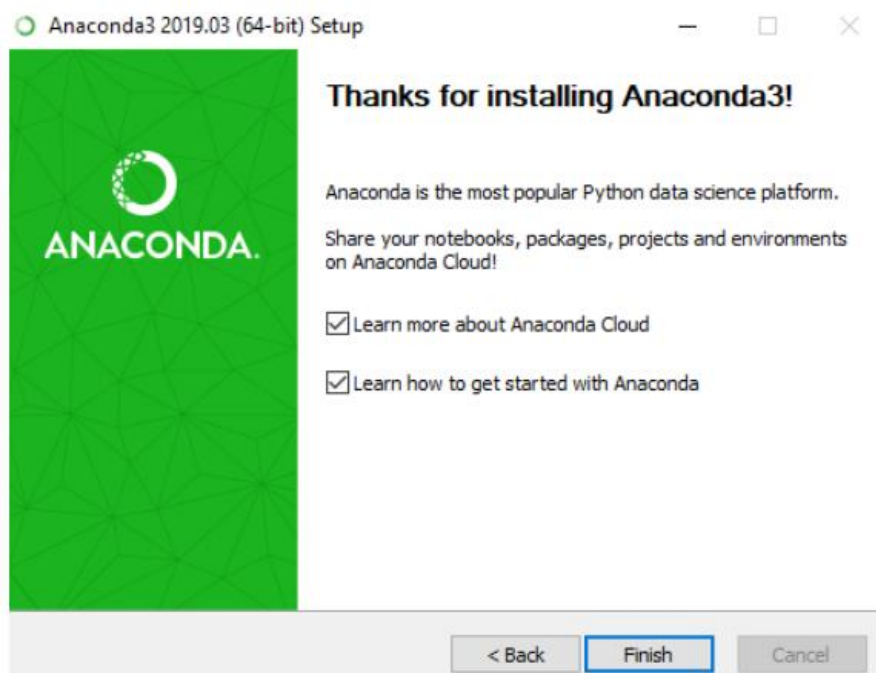


Рис.5. Окно завершения установки Anaconda

После завершения установки для запуска экосистемы Anaconda, необходимо в меню Пуск открыть Anaconda Navigator. Если программа запустится, то установка завершена успешно.

**В состав Anaconda входят пакеты, которые будут использоваться в следующих работах:*

- удобная среда разработки Spyder для ввода и запуска программ на Python;
- удобная среда выполнения кода в интерактивном режиме Jupyter Notebook.

4- Для запуска пакета экосистемы Anaconda необходимо в меню Пуск открыть **Anaconda Navigator**. В открывшемся окне навигационной панели отобразятся все компоненты (пакеты) экосистемы (рис. 6).

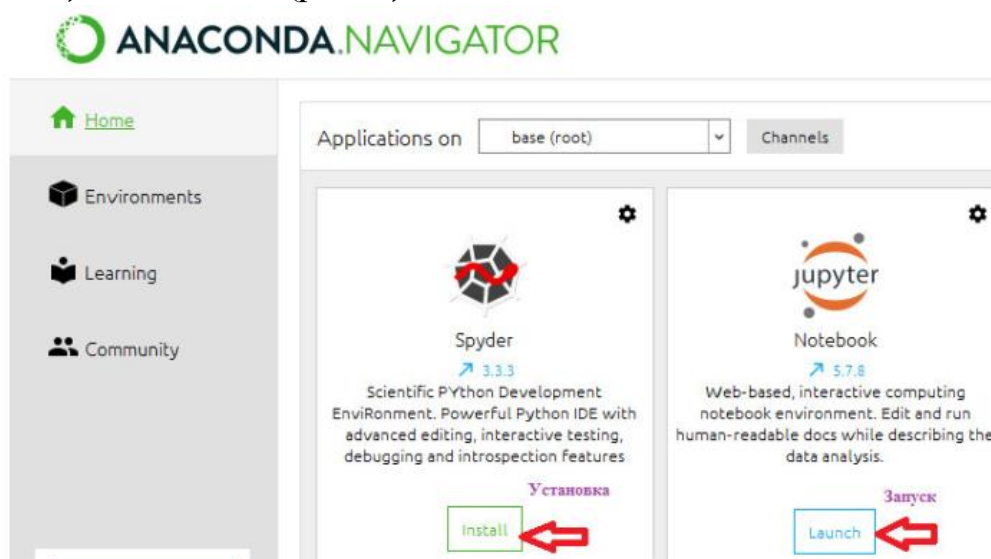


Рис. 6 Навигационная панель платформы Anaconda

Компоненты (пакеты), у которых в нижней части отображается кнопка *Install*, требуют установки. Установка будет выполнена при нажатии на данную кнопку. Компоненты (пакеты), у которых в нижней части отображается кнопка *Launch*, готовы к использованию, и после нажатия данной кнопки будут запущены.

Задание 3. Установить и настроить платформу Jupyter Notebook

Jupyter Notebook является графической веб-оболочкой для интерактивной оболочки IPython языка программирования Python. В Jupyter Notebook можно разрабатывать, документировать и выполнять приложения на языке Python, отображать результаты вычислений с медиа представлением (схемы, графики). Введенный код, комментарии и медиаданные сохраняются в файлах (ноутбуках). Запустить Jupyter Notebook можно из экосистемы Anaconda, нажав кнопку Launch.

Подробное руководство по использованию Jupyter Notebook можно найти на официальном сайте. Ниже даются основные правила работы.

После запуска программы она откроется в браузере (рис. 7).

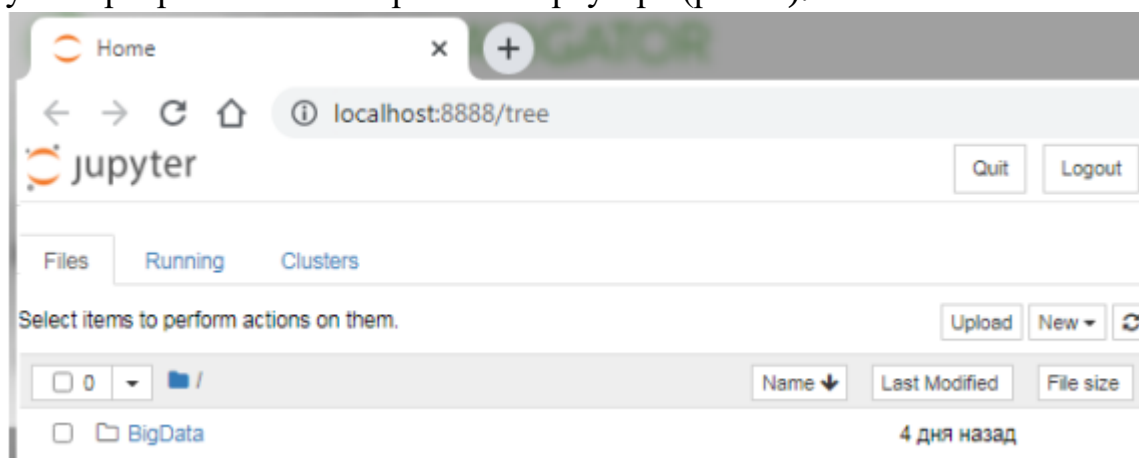


Рис.7 Стартовое окно программы Jupyter Notebook

В начале работы необходимо создать папку для хранения файлов практических работ. Для этого нажмите на кнопку *New* в правой части экрана и выберите в выпадающем списке *Folder* (рис.8).

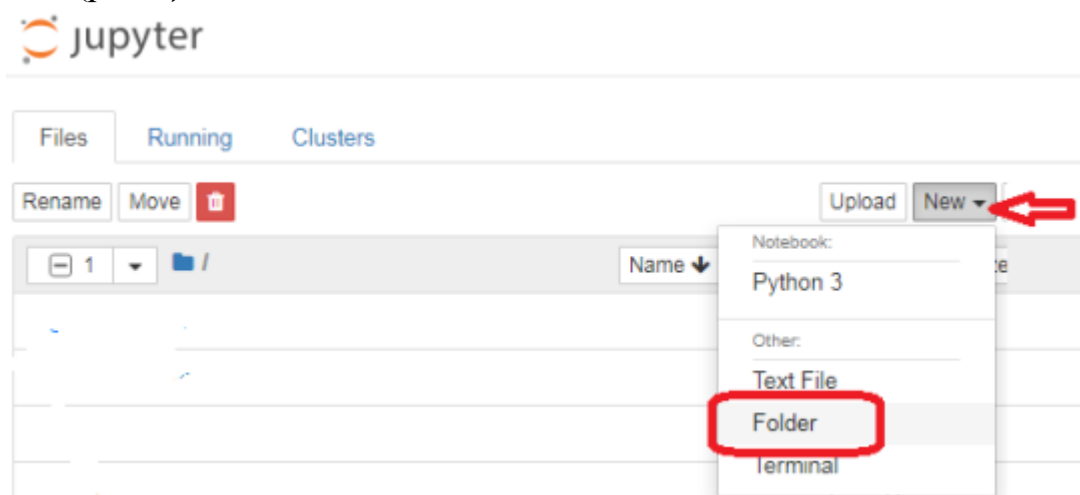


Рис. 8 Создание папки для сохранения файлов-блокнотов

По умолчанию папке присваивается имя *Untitled folder*. Переименуйте ее, например, в *Практические работы*. Для этого поставьте галочку напротив имени папки и нажмите на кнопку *Rename* (рис.9а). В появившемся окне измените прежнее имя папки на новое и снова нажмите на кнопку *Rename* (рис.9б).

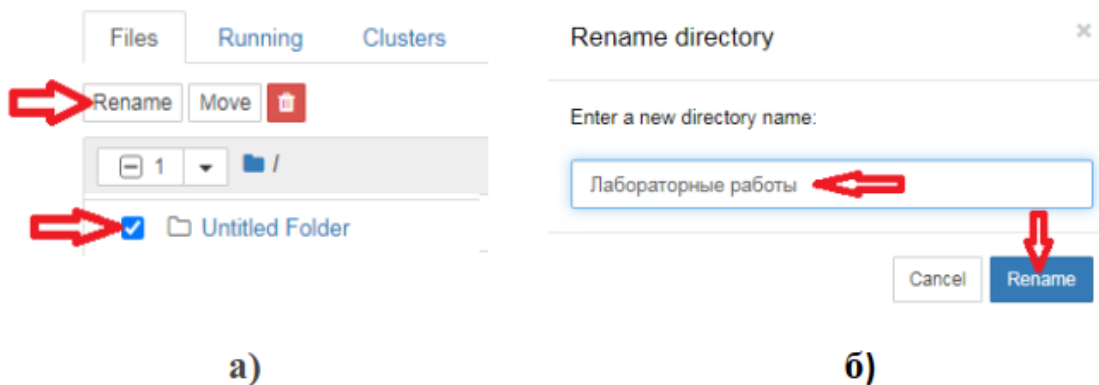


Рис.9 Переименование папки

Зайдите в эту папку и создайте в ней ноутбук (файл с будущим кодом), воспользовавшись той же кнопкой New, только на этот раз нужно выбрать *Python 3* (рис.10).

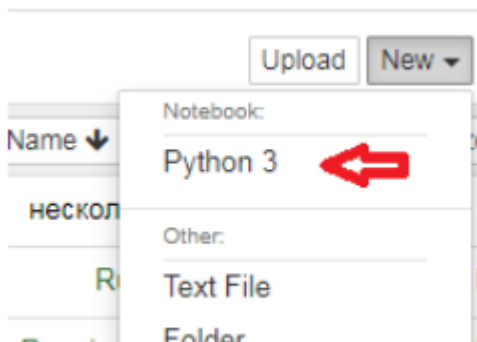


Рис. 10. Создание нового файла-блокнота

Новый файл откроется на новой вкладке браузера и ему будет присвоено имя *Untitled*. Файл также следует переименовать, выбрав пункт *File - Rename* или щелкнув по имени файла (рис.11а). В появившемся окне измените прежнее имя папки на новое и снова нажмите на кнопку *Rename* (рис.11б).

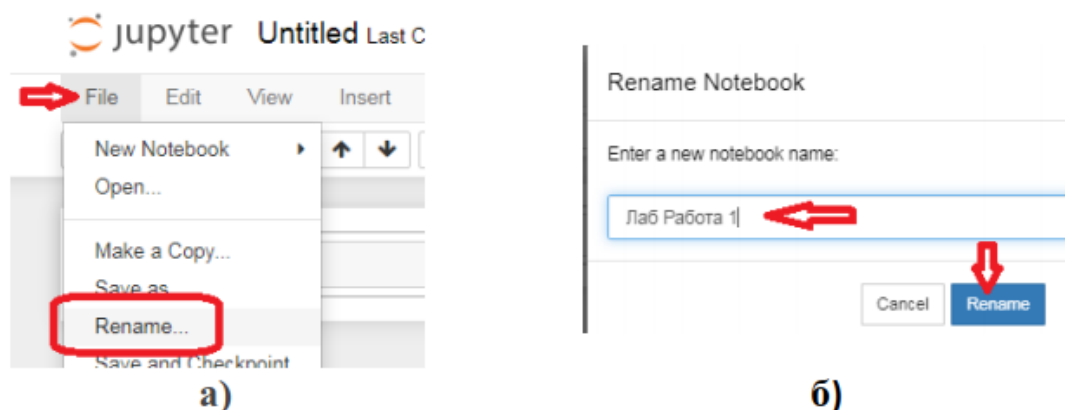


Рис.11 Переименование файла-блокнота

Код на языке Python или текст в нотации Markdown (произвольный текст для комментариев) нужно вводить в ячейки блокнота (рис.12).

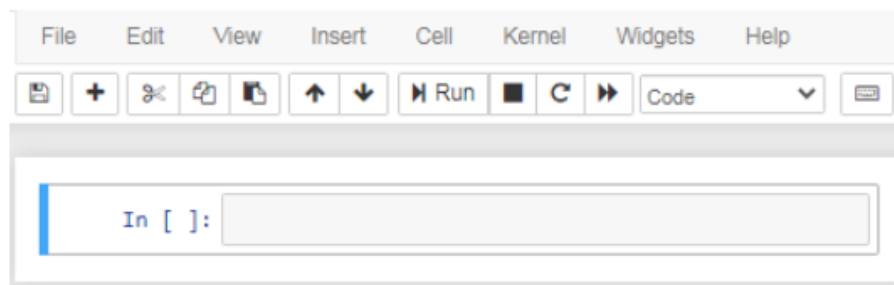


Рис. 12. Ячейка для ввода программного кода или комментариев

Если это код Python, то на панели инструментов в выпадающем списке нужно выставить свойство Code (рис.13а), а для комментариев Markdown (рис.13б).

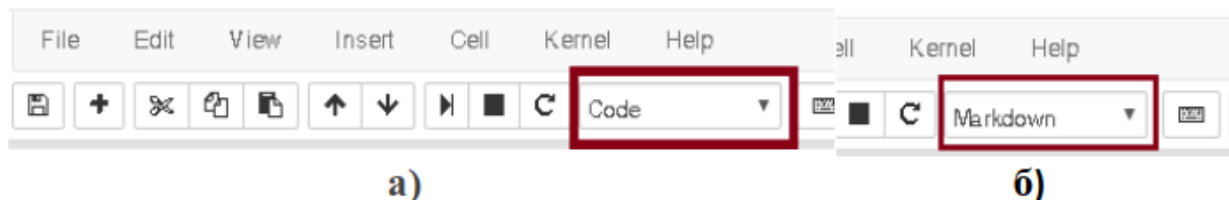


Рис. 13. Выбор типа вводимых данных

Для начала необходимо решить простую арифметическую задачу: вычислим произведение двух чисел. Вначале введем комментарий. Зададим свойство *Markdown*, в ячейке введем текст комментария *Первый пример*, а затем нажмите Ctrl+Enter или Shift+Enter или кнопку  на панели инструментов (рис. 14). В первом случаях будет просто добавлен комментарий в блокнот, а в других случаях после вводе комментария будет создана новая ячейка для ввода

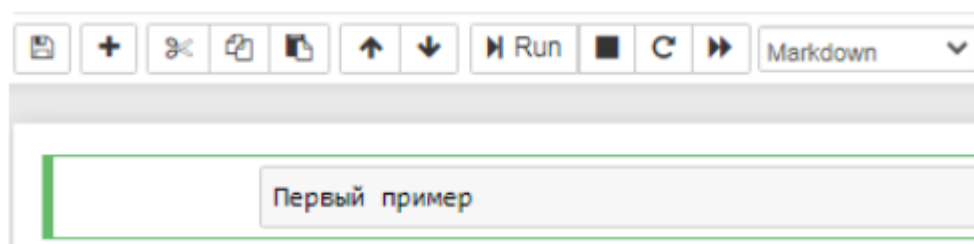
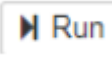


Рис. 14. Ввод комментария

Теперь введем программный код для вычисления произведения. Для этого зададим свойство вида вводимых данных Code, введем в ячейке $3 * 5$ нажмём кнопку  на панели инструментов (рис. 15).

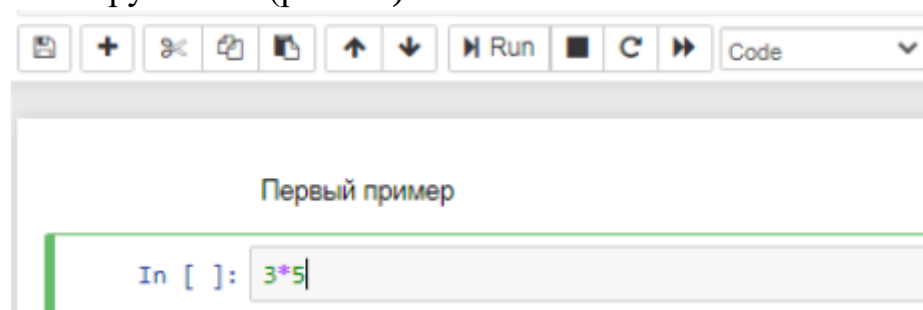


Рис. 15. Ввод программного кода

Интерпретатор Python выполнить введенный код, будет выведен результат и добавлена новая ячейка (рис. 16).

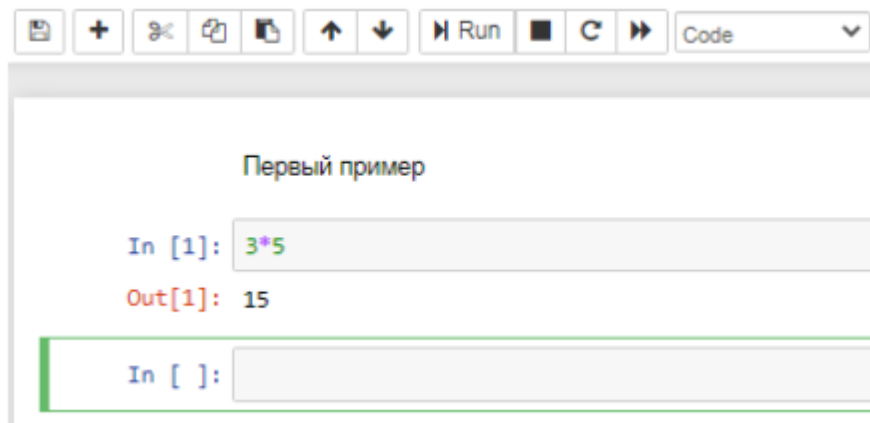


Рис. 16. Вывод результата

Из других элементов интерфейса можно выделить, панель меню (рис.17а) и расположенную под меню панель инструментов (рис.17б).

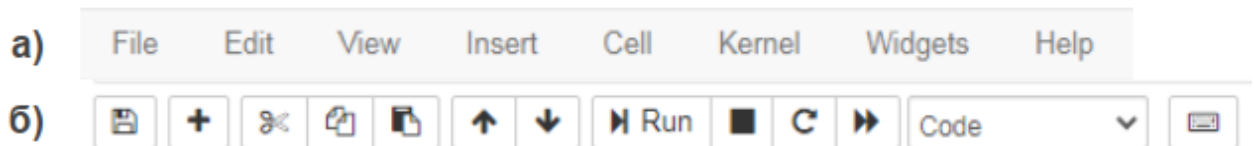


Рис. 17. Элементы интерфейса окна Jupyter Notebook

Программа может находиться в одном из двух режимов: режиме правки (Edit mode) и командном режиме (Command mode). Текущий режим отображается на панели меню в правой части, в режиме правки появляется изображение карандаша (рис.18), отсутствие этой иконки значит, что ноутбук находится в командном режиме.

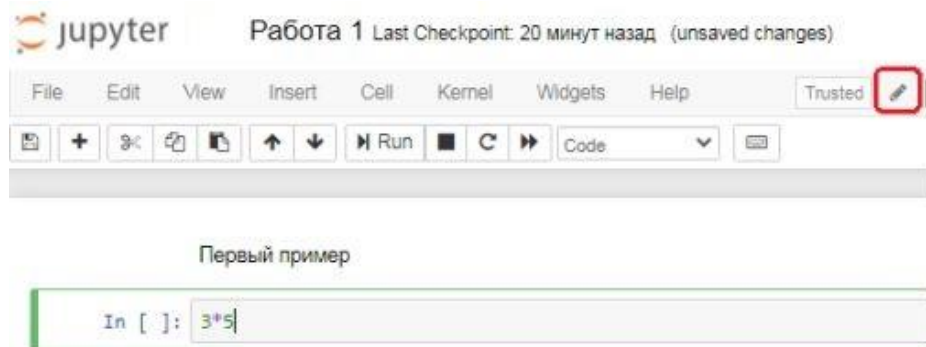


Рис.18 Программа в режиме правки

В самой правой части панели меню находится индикатор загруженности ядра Python. Если ядро находится в режиме ожидания, то индикатор представляет собой окружность (рис.19а). Если оно выполняет какую-то задачу, то изображение измениться на закрашенный круг, а на закладке страницы браузера отображается иконка песочных часов (рис. 19б).

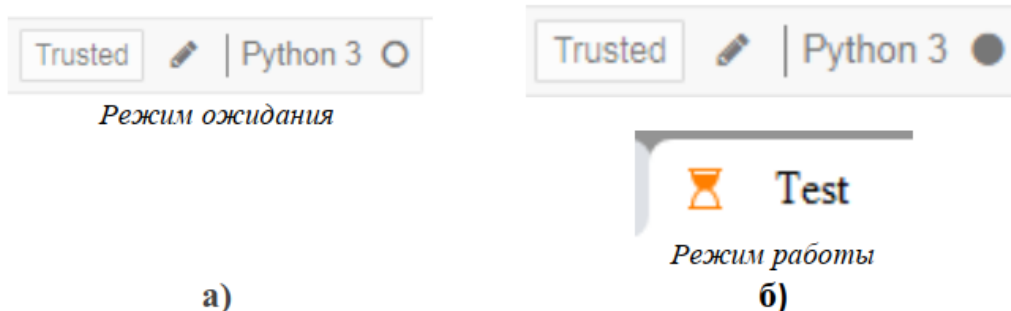


Рис 19. Индикаторы загруженности ядра Python

Если программа зависла, то можно прервать ее выполнение, выбрав на панели меню

пункт *Kernel -> Interrupt*.

Для добавления новой ячейки в середине блокнота используйте меню:

Insert->Insert Cell Above (выше текущей ячейки)

или

Insert->Insert Cell Below (ниже текущей ячейки).



Во втором случае можно также использовать кнопку панели

Задание 4. Необходимо организовать подготовку данных для построения модели (допустим модели классификации). В качестве данных выбран набор данных об ирисах Фишера.

Данный набор данных предназначен для построения модели классификации. Данные о 150 экземплярах ириса (рис. 1.1), по 50 экземпляров из трёх видов – Ирис щетинистый (*Iris setosa*), Ирис виргинский (*Iris virginica*) и Ирис разноцветный (*Iris versicolor*). Для каждого экземпляра измерялись четыре характеристики (в сантиметрах):

- 1) длина наружной доли околоцветника (sepal length);
- 2) ширина наружной доли околоцветника (sepal width);
- 3) длина внутренней доли околоцветника (petal length);
- 4) ширина внутренней доли околоцветника (petal width).

На основании этого набора данных требуется построить правило классификации, определяющее вид растения по данным измерений. Это задача многоклассовой классификации, так как имеется три класса – три вида ириса.



а)



б)



в)

Рисунок 1.1 – Внешний вид классифицируемых ирисов: а) *Iris setosa*; б) *Iris virginica*; в) *Iris versicolor*

Необходимо скачать набор данных из репозитория Center for Machine Learning and Intelligent Systems (необходим только один текстовый файл с данными измерений):

<http://archive.ics.uci.edu/ml/datasets/Iris>

Файл Iris.data при просмотре выглядит следующим образом (рис. 1.2):

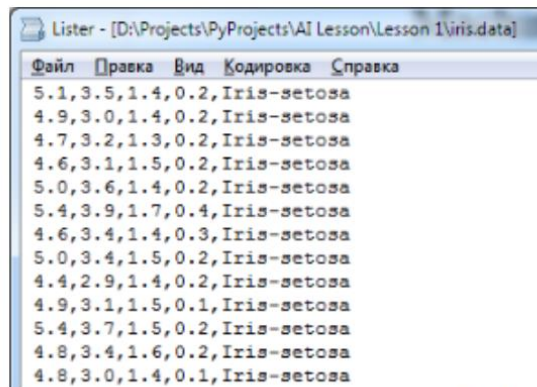
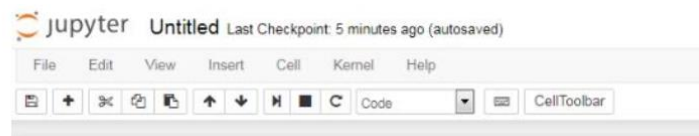


Рисунок 1.2 – Внешний вид данных файла Iris.data

Использовать текстовые редакторы для просмотра и анализа данных из определенных наборов – нерациональный вариант. Поэтому запустим Jupyter Notebook и начнем работать с загруженным набором с использованием среды Python. Используем метод `genfromtxt()` из пакета `scipy`.



Лабораторная работа 1

Исследование набора данных ирисы Фишера

```
In [36]: import numpy as np

data = np.genfromtxt("iris.data", delimiter=",")

print(data)
```

```
[[ 5.1  3.5  1.4  0.2 nan]
 [ 4.9  3.   1.4  0.2 nan]
 [ 4.7  3.2  1.3  0.2 nan]
 [ 4.6  3.1  1.5  0.2 nan]
 [ 5.   3.6  1.4  0.2 nan]
 [ 5.4  3.9  1.7  0.4 nan]
 [ 4.6  3.4  1.4  0.3 nan]
 [ 5.   3.4  1.5  0.2 nan]
 [ 4.4  2.9  1.4  0.2 nan]
 [ 4.9  3.1  1.5  0.1 nan]
 [ 5.4  3.7  1.5  0.2 nan]
 [ 4.8  3.4  1.6  0.2 nan]
 [ 4.8  3.   1.4  0.1 nan]
 [ 4.3  3.   1.1  0.1 nan]
 [ 5.8  4.   1.2  0.2 nan]
 [ 5.7  4.4  1.5  0.4 nan]
 [ 5.4  3.9  1.3  0.4 nan]
 [ 5.1  3.5  1.4  0.3 nan]
 [ 5.7  3.8  1.7  0.3 nan]
 [ 5.4  3.   1.5  0.2 nan]]
```

Рисунок 1.3 – Загрузка данных файла Iris.data

Метод `genfromtxt()` возвращает массив `numpy` (тип `numpy.ndarray`). Следует обратить внимание, что пятый столбец содержит неопределенные значения `numpy.NaN`.

Производить вывод всего источника данных – нерациональный путь. В реальных задачах данных может оказаться слишком много, поэтому чаще всего используют подвыборку данных для поверхностного обзора исследуемой обучающей выборки (рис. 1.4).

```
In [37]: print ( "Data type : ", type(data) )
print ( "Data shape : ", data.shape )
print ( data[:10] )

Data type : <class 'numpy.ndarray'>
Data shape : (150, 5)
[[ 5.1  3.5  1.4  0.2 nan]
 [ 4.9  3.   1.4  0.2 nan]
 [ 4.7  3.2  1.3  0.2 nan]
 [ 4.6  3.1  1.5  0.2 nan]
 [ 5.   3.6  1.4  0.2 nan]
 [ 5.4  3.9  1.7  0.4 nan]
 [ 4.6  3.4  1.4  0.3 nan]
 [ 5.   3.4  1.5  0.2 nan]
 [ 4.4  2.9  1.4  0.2 nan]
 [ 4.9  3.1  1.5  0.1 nan]]
```

Рисунок 1.4 – Начальное исследование Iris.data

Из представленного фрагмента видно, что data – это двумерный массив размером 150x5, или можно сказать, что это одномерный массив, каждый элемент которого также одномерный массив размером 5 элементов.

В рамках данной задачи необходимо получить значения пятого столбца. Для этого желательно использовать другой подход (рис. 1.5)

```
In [69]: data1 = np.genfromtxt("iris.data", delimiter=";", dtype=None)
print(data1.shape)
print(type(data1))
print(type(data1[0]))
print(type(data1[0][4]))
print(data1[:10])

(150,)
<class 'numpy.ndarray'>
<class 'numpy.void'>
<class 'numpy.bytes_'>
[(5.1, 3.5, 1.4, 0.2, b'Iris-setosa') (4.9, 3.0, 1.4, 0.2, b'Iris-setosa')
 (4.7, 3.2, 1.3, 0.2, b'Iris-setosa') (4.6, 3.1, 1.5, 0.2, b'Iris-setosa')
 (5.0, 3.6, 1.4, 0.2, b'Iris-setosa') (5.4, 3.9, 1.7, 0.4, b'Iris-setosa')
 (4.6, 3.4, 1.4, 0.3, b'Iris-setosa') (5.0, 3.4, 1.5, 0.2, b'Iris-setosa')
 (4.4, 2.9, 1.4, 0.2, b'Iris-setosa') (4.9, 3.1, 1.5, 0.1, b'Iris-setosa')]
```

Рисунок 1.5 – Загрузка данных разного типа в массив

Желательно (на первоначальных этапах изучения Python) проводить анализ типов различных значений. На рис. 1.6 представлен еще один вариант загрузки данных в массив numpy.ndarray.

```
In [70]: dt = np.dtype("f8, f8, f8, f8, U30")
data2 = np.genfromtxt("iris.data", delimiter=";", dtype=dt)
print(data2.shape)
print(type(data2))
print(type(data2[0]))
print(type(data2[0][4]))
print(data2[:10])

(150,)
<class 'numpy.ndarray'>
<class 'numpy.void'>
<class 'numpy.str_'>
[(5.1, 3.5, 1.4, 0.2, 'Iris-setosa') (4.9, 3.0, 1.4, 0.2, 'Iris-setosa')
 (4.7, 3.2, 1.3, 0.2, 'Iris-setosa') (4.6, 3.1, 1.5, 0.2, 'Iris-setosa')
 (5.0, 3.6, 1.4, 0.2, 'Iris-setosa') (5.4, 3.9, 1.7, 0.4, 'Iris-setosa')
 (4.6, 3.4, 1.4, 0.3, 'Iris-setosa') (5.0, 3.4, 1.5, 0.2, 'Iris-setosa')
 (4.4, 2.9, 1.4, 0.2, 'Iris-setosa') (4.9, 3.1, 1.5, 0.1, 'Iris-setosa')]
```

Рисунок 1.6 – Загрузка данных в массив с типом, определяемым пользователем

Было загружено 150 элементов данных, но даже при такой маленькой выборке невозможно что-либо сказать о наборе данных. Для получения дополнительной информации необходимо визуализировать загруженные данные. В нашем случае каждый элемент данных представлен вещественными признаками – это существенно

упрощает визуализацию (рис. 1.7). Но сложность заключается в том, что приходится работать с элементами 4-мерного пространства, поэтому строится не графическое представление распределения, а отдельные проекции.

Уже из графического распределения на рис. 1.7 видно, что тип ирисов *Setosa* хорошо отделяется. На данном графике представлено отображение в плоскости признаков ('Sepal Width', 'Sepal Length') Но исследователь имеет возможность построить столько графиков, сколько необходимо для глубокого анализа данных. Изменим ячейку In [72] в соответствии с листингом, представленным на рис. 1.8.

```
In [72]: import matplotlib as mpl
import matplotlib.pyplot as plt

# Данные из отдельных столбцов
sepal_length = [] # Sepal Length
sepal_width = [] # Sepal Width
petal_length = [] # Petal Length
petal_width = [] # Petal Width

# Выполняем обход всей коллекции data2
for dot in data2:
    sepal_length.append(dot[0])
    sepal_width.append(dot[1])
    petal_length.append(dot[2])
    petal_width.append(dot[3])

# Строим графики по проекциям данных
# Учтем, что каждые 50 типов ирисов идут последовательно
plt.figure(1)
setosa, = plt.plot(sepal_length[:50], sepal_width[:50], 'ro', label='Setosa')
versicolor, = plt.plot(sepal_length[50:100], sepal_width[50:100], 'g^', label='Versicolor')
virginica, = plt.plot(sepal_length[100:150], sepal_width[100:150], 'bs', label='Verginica')
plt.legend(bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0.)
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')

plt.show()
```

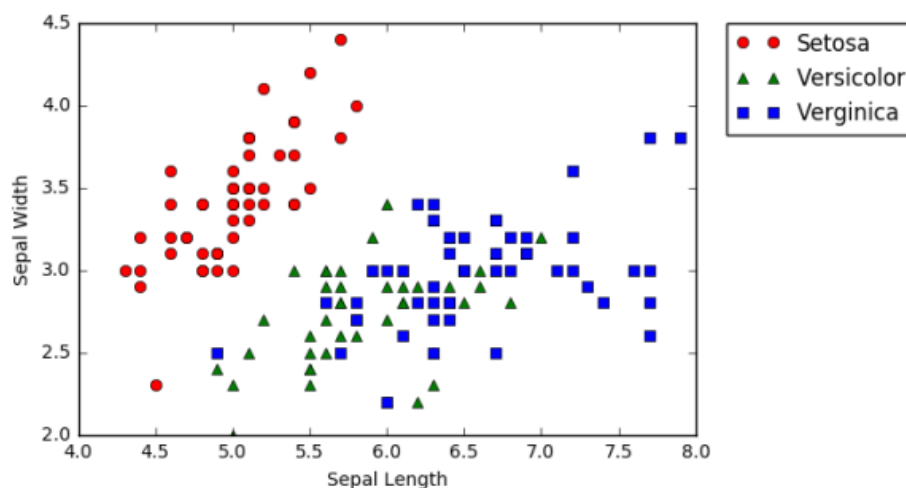


Рисунок 1.7 – Простейший анализ данных по графическому представлению

```
In [74]: import matplotlib as mpl
import matplotlib.pyplot as plt

# Данные из отдельных столбцов
sepal_length = [] # Sepal Length
sepal_width = [] # Sepal Width
petal_length = [] # Petal Length
petal_width = [] # Petal Width

# Выполняем обход всей коллекции data2
for dot in data2:
    sepal_length.append(dot[0])
    sepal_width.append(dot[1])
    petal_length.append(dot[2])
    petal_width.append(dot[3])

# Строим графики по проекциям данных
# Учитываем, что каждые 50 типов ирисов идут последовательно
plt.figure(1)
setosa = plt.plot(sepal_length[:50], sepal_width[:50], 'ro', label='Setosa')
versicolor = plt.plot(sepal_length[50:100], sepal_width[50:100], 'g^', label='Versicolor')
virginica = plt.plot(sepal_length[100:150], sepal_width[100:150], 'bs', label='Virginica')
plt.legend(bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0.)
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')

plt.figure(2)
setosa = plt.plot(sepal_length[:50], petal_length[:50], 'ro', label='Setosa')
versicolor = plt.plot(sepal_length[50:100], petal_length[50:100], 'g^', label='Versicolor')
virginica = plt.plot(sepal_length[100:150], petal_length[100:150], 'bs', label='Virginica')
plt.legend(bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0.)
plt.xlabel('Sepal Length')
plt.ylabel('Petal Length')

plt.figure(3)
setosa = plt.plot(sepal_length[:50], petal_width[:50], 'ro', label='Setosa')
versicolor = plt.plot(sepal_length[50:100], petal_width[50:100], 'g^', label='Versicolor')
virginica = plt.plot(sepal_length[100:150], petal_width[100:150], 'bs', label='Virginica')
plt.legend(bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0.)
plt.xlabel('Sepal Length')
plt.ylabel('Petal Width')

plt.show()
```

Рисунок 1.8 – Построение простейшего графика для отображения различных проекций данных

Вывод данного кода представлен на рис. 1.9.

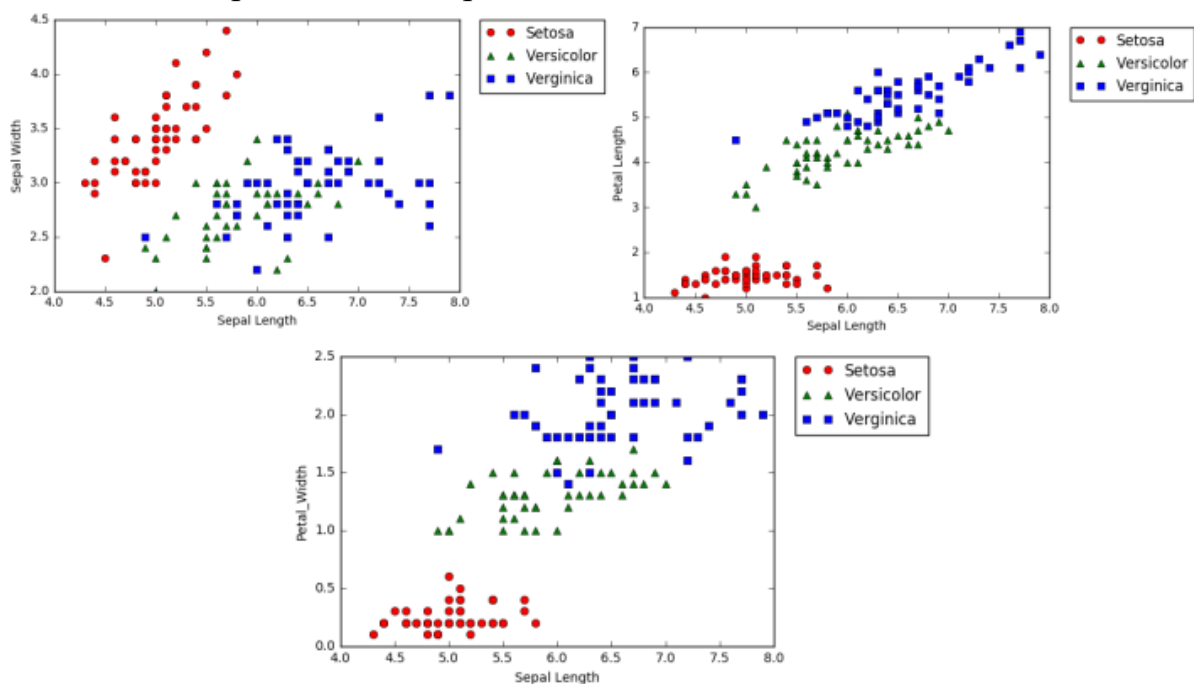


Рисунок 1.9 – Вывод графика для отображения различных проекций данных

Из графиков 1.9 видно, что множество *Setosa* хорошо отделимо, а множества *Versicolor* и *Virginica* представляют собой множества, разделение которых является

непростой задачей.

Перечень контрольных вопросов

- 1- Какие архитектуры нейронных сетей используются для обработки изображений?
- 2- В чём разница между классическим компьютерным зрением и распознаванием на основе глубокого обучения?
- 3- В каких сферах наиболее активно применяется распознавание изображений?
- 4- Что такое трансферное обучение? Когда и почему его целесообразно применять при распознавании изображений?
- 5- Пройдите тест по ссылке: <https://forms.yandex.ru/u/69ca57f895add5954b4673e8>